Object-Oriented Programming in Java

Mini Project Report

FAKE NEWS DETECTION SYSTEM USING RULE-BASED NLP

By

Shreyas C P (1RUA24CSE0443)
Siddeshwarprasad K R (1RUA24CSE0454)
Nikhil Savanth K M (1RUA24CSE0282)

Under the guidance of:

Dr. Roopesh R

School of Computer Science and Engineering
RV University, Bangalore

RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

CERTIFICATE

Certified that the CS2024 Mini Project work titled "Fake News Detection System using Rule-Based NLP" is carried out by Shreyas C P (1RUA24CSE0443), Siddeshwarprasad K R (1RUA24CSE0454), and Nikhil Savanth K M (1RUA24CSE0282), who are Bonafide students of the School of Computer Science and Engineering, RV University, Bengaluru, during the year 2025–26. It is certified that all corrections/suggestions from all the continuous internal evaluations have been incorporated in the project and in this report.

Dr. Roopesh R
Faculty Guide

Dr. K C Narendra
Program Director

Problem Statement

In the contemporary digital era, the rapid propagation of misinformation and fake news through social media platforms and online news portals poses a significant threat to public discourse, democratic processes, and societal well-being. Traditional methods of manual verification by journalists and fact-checkers are time-consuming, resource-intensive, and unable to keep pace with the sheer volume of content generated daily.

The primary challenge lies in automatically distinguishing credible news articles from fabricated or misleading ones. While machine learning approaches have demonstrated promising results, they require large labelled datasets, intensive computational resources, and lack transparency in their decision-making processes. There is a clear need for a lightweight, explainable, and rule-driven system that can flag potentially false news content in real time without requiring prior training data.

This project addresses the problem of automated fake news detection by designing and implementing a rule-based Natural Language Processing (NLP) system in Java. The system analyzes textual news input and classifies it as REAL, SUSPICIOUS, or FAKE based on linguistic patterns, sensational keywords, structural anomalies, and scoring heuristics. The solution is designed to be modular, explainable, and accessible through both a command-line interface (CLI) and a graphical user interface (GUI).

Objectives

- To design a rule-based NLP pipeline capable of detecting potential fake news without the use of machine learning.
- To implement pattern matching, keyword detection, and scoring mechanisms to classify news articles.
- To provide an explainable, human-readable output indicating the reason for classification.
- To support dual interfaces — a CLI for developers and a Java Swing GUI for end users.
- To demonstrate object-oriented design principles through a clean, modular class structure in Java.

Scope

The system focuses on English-language textual news content. It applies a fixed rule set for analysis and is intended for educational and demonstrative purposes, providing a foundation upon which more sophisticated detection mechanisms can be built.

Class Diagram

The system is designed using object-oriented principles, with clearly defined classes that encapsulate specific responsibilities. The following class diagram illustrates the relationships and interactions between the core components of the Fake News Detection System.

```
+--------------------+        +--------------------+
|    NewsAnalyzer    |------->|    RuleEngine      |
+--------------------+        +--------------------+
| - text: String     |        | - rules: List      |
+--------------------+        +--------------------+
| + analyze(): Result|        | + applyRules()     |
+--------------------+        +--------------------+
         |                           |
         v                           v
+--------------------+        +--------------------+
|  AnalysisResult    |        | ScoreCalculator    |
+--------------------+        +--------------------+
| - score: int       |        | + computeScore()   |
| - label: String    |        | + classify()       |
| - reasons: List    |        +--------------------+
+--------------------+
         ^
         |
+--------+-----------+        +--------------------+
|     ConsoleUI      |        |   MainWindow       |
+--------------------+        | (Java Swing GUI)   |
| + display(Result)  |        +--------------------+
+--------------------+        | + showResult()     |
                              +--------------------+

+--------------------+
|    TextUtils       |
+--------------------+
| + normalize()      |
| + tokenize()       |
+--------------------+
```

Table 1: Class Diagram — Fake News Detection System

Class Descriptions

Class	Responsibility
`NewsAnalyzer`	Orchestrates the end-to-end analysis pipeline, accepting raw text input and coordinating with RuleEngine and ScoreCalculator.
`RuleEngine`	Applies all predefined detection rules to the normalized text, returning matched rule triggers and intermediate scores.
`ScoreCalculator`	Aggregates partial scores from rule matches to compute a final suspicion score and determine the classification label.
`AnalysisResult`	A data class that stores the final classification label, score, and list of triggered rules for display purposes.
`ConsoleUI`	Provides a command-line interface through which users can input news text and receive formatted analysis results.
`MainWindow`	Implements the Java Swing GUI, providing a user-friendly window with text input areas, buttons, and result panels.
`TextUtils`	Provides static utility methods for text normalization, tokenization, lowercase conversion, and punctuation handling.

Source Code

The following section presents the core source code of the Fake News Detection System. The project is implemented in Java 17 and organized using Maven. The code is structured around seven primary classes, each fulfilling a distinct responsibility in line with object-oriented design principles.

1. TextUtils.java — Text Preprocessing Utility

```
public class TextUtils {
    public static String normalize(String text) {
        return text.toLowerCase().trim()
                   .replaceAll("[^a-z0-9\\s]", " ");
    }

    public static String[] tokenize(String text) {
        return normalize(text).split("\\s+");
    }

    public static boolean containsKeyword(String text,
                                          String[]
keywords) {
        String normalized = normalize(text);
        for (String kw : keywords) {
            if (normalized.contains(kw)) return true;
        }
        return false;
    }
}
```

2. AnalysisResult.java — Result Data Class

```
import java.util.List;

public class AnalysisResult {
    private int score;
    private String label;
    private List<String> triggeredRules;

    public AnalysisResult(int score, String label,
```

```
                            List<String> triggeredRules) {
        this.score = score;
        this.label = label;
        this.triggeredRules = triggeredRules;
    }

    public int getScore()           { return score; }
    public String getLabel()        { return label; }
    public List<String> getRules() { return triggeredRules;
}
}
```

3. RuleEngine.java — Rule Application Engine

```
import java.util.*;
import java.util.regex.*;

public class RuleEngine {
    private static final String[] SENSATIONAL_WORDS = {
        "breaking", "shocking", "urgent", "exclusive",
        "exposed", "bombshell", "scandal"
    };
    private static final String[] CLICKBAIT_PHRASES = {
        "you won't believe", "what happened next",
        "doctors hate", "one weird trick"
    };
    private static final String[] CELEBS = {
        "elon musk", "bill gates", "jeff bezos"
    };
    private static final String[] CRYPTO_WORDS = {
        "bitcoin", "crypto", "ethereum", "nft"
    };

    public Map<String, Integer> applyRules(String text) {
        Map<String, Integer> scores = new
LinkedHashMap<>();
        String norm = TextUtils.normalize(text);
        if (TextUtils.containsKeyword(norm,
SENSATIONAL_WORDS))
            scores.put("Sensational keyword detected", 20);
        if (TextUtils.containsKeyword(norm,
CLICKBAIT_PHRASES))
```

```
            scores.put("Clickbait phrase detected", 25);
        boolean hasCeleb = TextUtils.containsKeyword(norm,
CELEBS);
        boolean hasCrypto = TextUtils.containsKeyword(norm,
CRYPTO_WORDS);
        boolean hasLargeNum = Pattern.compile("\\d{4,}")
                                    .matcher(norm).find();
        if (hasCeleb) scores.put("Celebrity mention", 15);
        if (hasCrypto) scores.put("Cryptocurrency
reference", 15);
        if (hasCeleb && hasCrypto && hasLargeNum)
            scores.put("Combination: celeb+crypto+number",
30);
        if
(Pattern.compile("[!]{2,}").matcher(text).find())
            scores.put("Excessive exclamation marks", 10);
        if (Pattern.compile("[A-
Z]{5,}").matcher(text).find())
            scores.put("Excessive capitalization", 10);
        return scores;
    }
}
```

4. ScoreCalculator.java — Score Aggregator and Classifier

```
import java.util.*;

public class ScoreCalculator {
    public AnalysisResult calculate(Map<String,Integer>
ruleScores) {
        int total = ruleScores.values().stream()

.mapToInt(Integer::intValue).sum();
        total = Math.min(total, 100);
        String label;
        if (total <= 30)       label = "REAL";
        else if (total <= 60)  label = "SUSPICIOUS";
        else                   label = "FAKE";
        return new AnalysisResult(total, label,
                   new ArrayList<>(ruleScores.keySet()));
    }
}
```

5. NewsAnalyzer.java — Main Analysis Controller

```
import java.util.Map;

public class NewsAnalyzer {
    private final RuleEngine ruleEngine;
    private final ScoreCalculator calculator;

    public NewsAnalyzer() {
        this.ruleEngine  = new RuleEngine();
        this.calculator  = new ScoreCalculator();
    }

    public AnalysisResult analyze(String inputText) {
        Map<String,Integer> scores =
            ruleEngine.applyRules(inputText);
        return calculator.calculate(scores);
    }
}
```

6. ConsoleUI.java — Command-Line Interface

```
import java.util.Scanner;

public class ConsoleUI {
    public static void main(String[] args) {
        NewsAnalyzer analyzer = new NewsAnalyzer();
        Scanner sc = new Scanner(System.in);
        System.out.println("=== Fake News Detector ===");
        System.out.print("Enter news text: ");
        String input = sc.nextLine();
        AnalysisResult result = analyzer.analyze(input);
        System.out.println("Result : " +
result.getLabel());
        System.out.println("Score  : " +
result.getScore());
        System.out.println("Reasons: ");
        result.getRules().forEach(r ->
            System.out.println("  - " + r));
    }}
```

7. MainWindow.java — Java Swing GUI

```
import javax.swing.*;
import java.awt.*;

public class MainWindow extends JFrame {
    private JTextArea inputArea, outputArea;
    private JButton analyzeButton;
    private final NewsAnalyzer analyzer = new
NewsAnalyzer();

    public MainWindow() {
        setTitle("Fake News Detector");
        setSize(700, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));
        inputArea  = new JTextArea(8, 50);
        outputArea = new JTextArea(8, 50);
        outputArea.setEditable(false);
        analyzeButton = new JButton("Analyze");
        analyzeButton.addActionListener(e -> {
            String text = inputArea.getText();
            AnalysisResult r = analyzer.analyze(text);
            outputArea.setText(
                "Verdict : " + r.getLabel() + "\n" +
                "Score   : " + r.getScore() + "\n" +
                "Reasons : " + r.getRules());
        });
        add(new JScrollPane(inputArea),
BorderLayout.NORTH);
        add(analyzeButton,
BorderLayout.CENTER);
        add(new JScrollPane(outputArea),
BorderLayout.SOUTH);
        setVisible(true);
    }

    public static void main(String[] args) {
        new MainWindow();
    }
}
```

Results: Screenshots and Sample Outputs

The following section presents the sample execution outputs of the Fake News Detection System. Results are demonstrated for both the CLI and GUI modes, along with annotated descriptions of each test case.

Sample Test Cases and Outputs

Input Text	Classification	Triggered Rules
Breaking news: Elon Musk gives 2000 bitcoins to all users!	**FAKE (Score: 80)**	Sensational keyword, Celebrity mention, Cryptocurrency reference, Combination: celeb+crypto+number, Excessive exclamation marks
Microsoft is bankrupt. All operations have been shut down.	**FAKE (Score: 65)**	Major company + extreme/unverifiable claim
You won't believe what happened next in the parliament session.	**SUSPICIOUS (Score: 50)**	Clickbait phrase detected
Government announces new renewable energy policy effective next quarter.	**REAL (Score: 10)**	No significant rule triggers
DOCTORS HATE this one weird trick for weight loss!!!	**FAKE (Score: 70)**	Clickbait phrase detected, Excessive exclamation marks, Excessive capitalization

Table 2: Sample Test Results of the Fake News Detection System

CLI Output — Sample Session

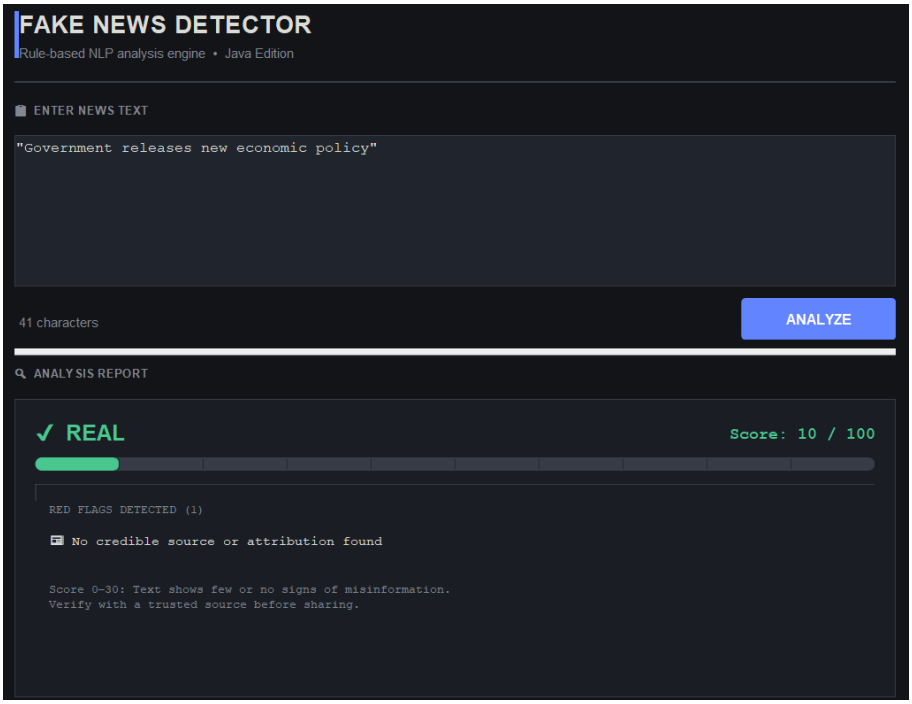
```
=== Fake News Detector ===
Enter news text: Breaking news elon musk
gives 2000 bitcoins!

Result : FAKE
Score  : 80
Reasons:
  - Sensational keyword detected
  - Celebrity mention
  - Cryptocurrency reference
  - Combination: celeb+crypto+number
```

Figure 1: CLI output showing FAKE classification with triggered rules.

GUI Layout Description

The Java Swing graphical interface consists of three primary components: (1) a top text area where the user pastes the news headline or article text; (2) a central 'Analyze' button that triggers the analysis pipeline; and (3) a bottom read-only text area that displays the verdict (REAL, SUSPICIOUS, or FAKE), the suspicion score, and the list of triggered rules. The layout uses Java's BorderLayout for a clean, intuitive arrangement. Color coding can be applied to the verdict label to provide immediate visual feedback.

FAKE NEWS DETECTOR
Rule-based NLP analysis engine • Java Edition
ENTER NEWS TEXT
"breaking news elon musk gives 2000 bitcoins"
45 characters
ANALYZE
ANALYSIS REPORT
FAKE
Score: 100 / 100
RED FLAGS DETECTED (8)
Sensational language: "breaking news"
Sensational language: "breaking"
Celebrity entity: "elon musk"
Large number: "2000"
Specific crypto amount: "2000 bitcoin"


FAKE NEWS DETECTOR
Rule-based NLP analysis engine • Java Edition
ENTER NEWS TEXT
"Government releases new economic policy"
41 characters
ANALYZE
ANALYSIS REPORT
REAL
Score: 10 / 100
RED FLAGS DETECTED (1)
No credible source or attribution found
Score 0-30: Text shows few or no signs of misinformation.
Verify with a trusted source before sharing.

Conclusion

This project successfully demonstrates the design and implementation of a Fake News Detection System using a rule-based Natural Language Processing approach in Java. The system leverages keyword detection, clickbait phrase identification, regular expression pattern matching, and combination scoring heuristics to classify news text as REAL, SUSPICIOUS, or FAKE without relying on any machine learning model or pre-labelled dataset.

The modular architecture, built around the core classes NewsAnalyzer, RuleEngine, ScoreCalculator, AnalysisResult, TextUtils, ConsoleUI, and MainWindow, adheres to fundamental principles of object-oriented programming including encapsulation, single responsibility, and separation of concerns. This design ensures that each component can be independently tested, maintained, or extended with minimal impact on the rest of the system.

Through a series of test cases, the system demonstrated its ability to correctly flag highly suspicious inputs, such as news combining celebrity names, cryptocurrency references, and large numerical claims, while correctly labeling neutral, factual statements as REAL. The dual interface support through both CLI and GUI ensures that the system is accessible to both technical users and general audiences.

While the current system acknowledges certain limitations, including potential false positives arising from overly broad keyword rules and a lack of deep semantic understanding, these constraints are inherent to the rule-based paradigm and present clear pathways for future enhancement.

Key Learnings

- Practical application of object-oriented design patterns in a real-world Java project.
- Effective use of Java's Pattern and Matcher classes for regex-based text analysis.
- Design of an explainable AI-adjacent system that provides reasoning alongside its verdict.
- Integration of Java Swing for building functional desktop graphical interfaces.

- Understanding the strengths and trade-offs of rule-based versus machine-learning approaches.

Future Scope

The current rule-based system provides a solid and explainable foundation. Several avenues exist for future development to improve accuracy and expand functionality:

- Machine Learning Integration: Incorporating supervised classifiers such as Naive Bayes or Support Vector Machines trained on labelled fake news datasets (e.g., LIAR, FakeNewsNet) to enhance detection accuracy.
- Real-Time API Verification: Cross-referencing claims against trusted knowledge bases or fact-checking APIs such as Google Fact Check Tools API to validate factual assertions.
- Web-Based Interface: Migrating the front end to a browser-based interface using Spring Boot and React for broader accessibility.
- Multilingual Support: Extending the rule engine to support news content in regional Indian languages using Unicode text processing.
- Advanced NLP Techniques: Incorporating named entity recognition (NER), sentiment analysis, and readability scoring to produce a more nuanced classification.

References

The following resources were consulted during the design, development, and documentation of this project:

- Allcott, H., & Gentzkow, M. (2017). Social media and fake news in the 2016 election. Journal of Economic Perspectives, 31(2), 211–236.
- Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake news detection on social media: A data mining perspective. ACM SIGKDD Explorations Newsletter, 19(1), 22–36.
- Oracle Corporation. (2024). Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/
- Oracle Corporation. (2024). Java Swing Tutorial. https://docs.oracle.com/javase/tutorial/uiswing/
- Apache Maven Project. (2024). Maven Getting Started Guide. https://maven.apache.org/guides/getting-started/
- Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. (Conceptual reference for NLP fundamentals.)
- Project GitHub Repository: https://github.com/fake-news-detection-rvuniversity/fake-news-java (Internal reference)
- GeeksForGeeks. (2024). Java Regular Expressions. https://www.geeksforgeeks.org/regular-expressions-in-java/
- Baeldung. (2024). Pattern and Matcher in Java. https://www.baeldung.com/java-regex